FALCA: Falsification-Aware Reinforcement Learning with Hierarchical Credit Assignment for Long-Horizon Deep Search Agents

Zijun Dou[1,2*], **Kun Luo**[2,3*], **Chaofan Li**[1], **Shuqi Lu**[1], **Zheng Liu**[1†]

[1] BAAI, [2] THU, [3] CASIA

{royz.dou,luokun695,zhengliu1026}@gmail.com

Abstract

Long-horizon deep search requires LLM agents to iteratively decompose complex information needs, gather evidence from external sources, and maintain a reliable research state under multiple constraints. A central challenge is that early mistakes can be amplified over the search trajectory: agents may anchor on an incorrect candidate, collect partially supporting evidence, overlook contradictions, and accumulate errors that misguide subsequent reasoning. We propose **FALCA**, a falsification-aware framework that formulates deep search as a propose–falsify process. FALCA adopts a collaborative Searcher–Falsifier architecture, where the Searcher proposes candidate answers or sub-answers, while independent Falsifiers actively seek counterevidence, check unmet constraints, identify conflicts, and produce concise falsification reports for research-state revision. To strengthen this loop, we introduce falsification-aware training, combining SFT warmup on falsification trajectories with agentic RL based on parallel rollouts and hierarchical credit assignment over research stages, state transitions, and actions. Experiments on five search benchmarks show that FALCA improves long-horizon deep search performance over backbone and harness ablations, while reducing confirmation bias and error accumulation through more effective falsification and research-state revision.

1 Introduction

Deep search requires large language model (LLM) agents to decompose complex, multi-constraint queries, retrieve and synthesize evidence from heterogeneous and potentially conflicting sources (Xia et al. 2025; Zheng et al. 2025; Press et al. 2023). Unlike single-step retrieval-augmented generation, deep search requires agents to continuously refine candidate hypotheses over long-horizon interactions, verify task constraints, resolve conflicting evidence, and update the evolving research context (Lewis et al. 2020; He et al. 2025). Therefore, long-horizon deep search demands not only stronger retrieval capability but also the ability to maintain a reliable and consistent research state throughout an iterative search process, making it a key capability for advancing LLMs from conversational assistants toward autonomous knowledge-exploration agents (Yao et al. 2026).

However, the difficulty of long-horizon deep search lies not merely in executing more search steps. Since each step depends on and modifies the research state, early mistakes can be amplified as the trajectory unfolds (Xia et al. 2026). We identify two sequential, mutually reinforcing failure modes underlying this process. First, **confirmation bias** leads the agent to repeatedly seek evidence for an early candidate while rarely attempting to falsify it or test its underlying constraints, allowing incorrect hypotheses to persist. These incorrect candidates, weakly supported claims, and unresolved contradictions are then incorporated into the research context, causing **error accumulation** that misguides subsequent search and reasoning. The corrupted research state further reinforces the agent's search bias, forming a self-reinforcing error loop. Thus, the challenge for deep search agents is not simply to search deeper, but to break this error loop by maintaining, questioning, and revising a reliable research state.

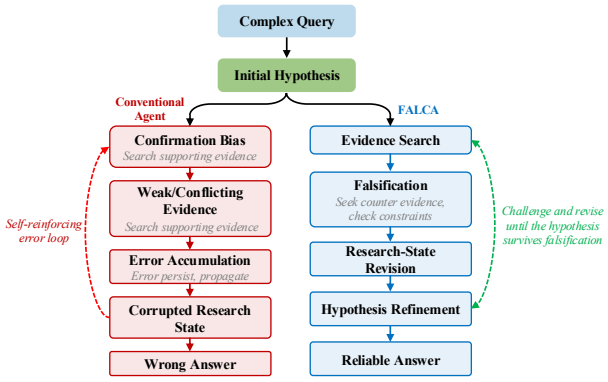


Figure 1: **Motivation for FALCA**. Conventional agents may fall into a self-reinforcing error loop, where confirmation bias leads to error accumulation and corrupts the research state. FALCA breaks this loop through iterative falsification, research-state revision, and hypothesis refinement.

Existing agents are typically optimized through trajectory imitation or final-answer rewards to search deeper along promising directions, but are rarely trained to challenge hypotheses, seek counterevidence, resolve conflicts, or revise the research state. This is problematic because long-horizon deep search resembles a constrained research process rather

*These authors contributed equally.

†Corresponding author.

Homepage: https://douzj-llm.github.io/FALCA/

than a linear retrieve-and-answer pipeline. The agent must propose candidates, test them against heterogeneous evidence, eliminate those that violate task constraints, and revise intermediate conclusions when conflicts emerge. Reliable progress therefore requires not only collecting supporting evidence but also identifying evidence that would invalidate current hypotheses, echoing the role of falsification in scientific inquiry (Popper 1963). We thus formulate long-horizon deep search as a **propose–falsify** process, in which the agent periodically alternates among *hypothesis candidate generation*, *active falsification*, and *research-state revision*. Under this view, falsification is not post-hoc verification but a context-management mechanism for checking whether the current research direction and candidate hypotheses remain valid before the agent searches deeper.

In this work, we propose **FALCA**, a falsification-based agentic framework for long-horizon deep search. FALCA adopts a collaborative Searcher–Falsifier architecture. The Searcher decomposes the problem, gathers evidence from external sources, maintains the research context, and proposes candidate answers. At periodic checkpoints, these candidates are passed to multiple Falsifier agents, each operating in an independent context to adversarially stress-test the candidate by seeking counterevidence, checking unmet constraints, identifying hidden conflicts, and suggesting revised search directions. Each Falsifier returns a concise falsification report summarizing the candidate's reliability, relevant evidence, unresolved or violated constraints, and recommended next steps. These reports are accumulated as a compact historical research state and fed back to the Searcher, which refreshes its working context and continues the propose–falsify loop until the information need is resolved.

To strengthen this propose–falsify research loop, we adopt a two-stage paradigm consisting of SFT warmup followed by falsification-aware agentic RL. The model is first trained on falsification-aware trajectories to acquire basic capabilities for search, verification, counterevidence seeking, and context revision. We then perform parallel rollouts for each problem to construct diverse trajectories that include successful, unsuccessful, and behaviorally distinct falsification paths. Through hierarchical reward modeling and credit assignment, trajectory-level task feedback is further decomposed across research stages, state transitions, and individual actions. Compared with agentic RL methods that rely primarily on final outcome rewards, FALCA can reinforce search, falsification, and conflict-resolution behaviors that genuinely improve the research state, while avoiding indiscriminate reinforcement of redundant steps in successful trajectories or uniform penalization of locally valuable actions in failed trajectories. Finally, we optimize the policy using a PPO-style clipped objective with shaped token-level advantages and KL regularization to the SFT reference policy.

Our contributions are threefold:

- We formulate long-horizon deep search as a falsification-centered propose–falsify process and instantiate it with a collaborative Searcher–Falsifier architecture, which periodically challenges candidate hypotheses and revises the research state to mitigate confirmation bias and error accumulation.
- We develop a falsification-aware agentic RL framework that combines SFT warmup, parallel rollouts, and hierarchical credit assignment over trajectories, research stages, and actions, enabling the policy to learn search, falsification, and state-revision behaviors that improve the evolving research state.
- We conduct extensive experiments on five search benchmarks, showing that FALCA improves answer correctness and long-horizon search behavior, with ablations validating the contributions of falsification, research-state revision, and hierarchical credit assignment.

2 Related Work

Long-Horizon Search Agents. Retrieval-augmented generation grounds LLMs in external knowledge, while WebGPT, ReAct, and IRCoT extend this paradigm by interleaving reasoning with retrieval, browsing, and tool interaction (Lewis et al. 2020; Nakano et al. 2021; Yao et al. 2023; Trivedi et al. 2023). More recent work trains LLMs to determine when and how to search through multi-turn interaction, using outcome-based reinforcement learning in controlled retrieval environments or end-to-end optimization over the open web (Song et al. 2025; Jin et al. 2025; Chen et al. 2025b; Zheng et al. 2025). Complementary approaches study active context management for long-horizon information seeking (Yao et al. 2026). These methods improve search depth and adaptability, but primarily optimize information acquisition or final-answer quality; the evolving research state is seldom modeled as a collection of falsifiable hypotheses whose supporting evidence, contradictions, and unmet constraints must be explicitly maintained and revised.

Reflection, Verification, and Falsification. Related work improves model reliability through iterative self-feedback, episodic reflection, tool-grounded critique, or learned reflection signals (Madaan et al. 2023; Shinn et al. 2023; Gou et al. 2024; Asai et al. 2024). Web-agent methods incorporate reasoning behaviors such as reflection, lookahead, branching, and rollback (Hu et al. 2025). However, these mechanisms typically refine a generated output, recover from observed failures, or assess retrieved evidence, rather than treating falsification as a persistent search operation. In contrast, our method explicitly assigns a Falsifier to seek disconfirming evidence, test unresolved constraints, expose contradictions, and trigger research-state revision before final synthesis, thereby addressing confirmation bias throughout the trajectory rather than only verifying an already formed answer.

Agentic Reinforcement Learning and Credit Assignment. Search-oriented agents are commonly initialized through supervised imitation and optimized with trajectory-level rewards. Outcome-based RL can elicit multi-turn retrieval, planning, and self-correction (Song et al. 2025; Jin et al. 2025; Chen et al. 2025b; Zheng et al. 2025), but delayed rewards provide limited supervision for distinguishing critical verification and revision from redundant exploration. Recent methods address this issue through hierarchical group-relative advantages, trajectory decomposition, and agent-specific process reward models (Feng et al. 2025; Luo et al.

2025; Xi et al. 2025). Unlike generic action-quality or goal-progress supervision, FALCA assigns credit to research-state transitions, reinforcing actions that invalidate unsupported hypotheses, reconcile conflicting evidence, recover missing constraints, or improve research-state reliability.

3 Methodology

3.1 Problem Formulation

We formulate long-horizon deep search as a sequential decision process. Given a query q and answer y^*, the agent interacts with an external environment to generate:

$$\tau = (s_0, a_1, o_1, \cdots, a_T, o_T), \tag{1}$$

where s_t, a_t, o_t denote the research state, action, and observation. The goal is to learn a policy $\pi_\theta(a_t|s_t)$ for search, verification, and answer synthesis under limited budgets.

We further represent long trajectories at multiple granularities. A turn denotes one interaction step, while consecutive turns are grouped into segments:

$$\tau = (g_0, g_1, \cdots, g_K). \tag{2}$$

Segments serve as basic units for research state evolution, reward modeling, and credit assignment. For multi-hop tasks, we model latent sub-goals: $q \to e_1 \to e_2 = y^*$, where intermediate entity discovery provides process-level supervision.

3.2 Tool Design

To support long-horizon deep search, we equip the Searcher and Falsifier with a shared evidence environment. Both agents access the same retrieval and browsing tools to acquire consistent evidence, while maintaining different objectives and action spaces. The Searcher explores and maintains the research state, while the Falsifier seeks contradictions and returns structured feedback. Specifically, FALCA contains five tools:

- **Search**: Retrieves relevant documents from external sources using batched queries for evidence discovery.
- **Visit**: Summarizes retrieved webpages into goal-conditioned evidence.
- **Update Context**: Compresses long interaction histories into structured research states containing verified facts, unresolved constraints, and future plans.
- **Finish**: Terminates the Searcher trajectory by producing the final answer with supporting evidence.
- **Refute**: Enables the Falsifier to return structured adversarial reports, including rejected constraints, evidence, and recommended follow-up searches.

3.3 Falsification-Guided Research Trajectory Construction

Generative deep-search agents may prematurely accept plausible but weakly supported candidates and repeatedly propagate them into subsequent contexts, leading to confirmation bias and error accumulation. To mitigate this issue, we construct falsification-guided research trajectories in which candidate hypotheses proposed by the Searcher are actively challenged by the Falsifier before being incorporated into later research states.

At the k-th research stage, the Searcher generates a candidate set from the current research state S_k: $\mathcal{H}_k = \{h_k^1, \cdots, h_k^{M_k}\}$. For each candidate h_k^m, an independent Falsifier follows a contradiction-first strategy to search for counterevidence and returns a structured report through the `refute` tool: $F_k^m = (v_k^m, C_{\text{ref}}^m, C_{\text{unr}}^m, E_k^m, Q_{\text{next}}^m)$, where $v_k^m \in \{\texttt{reject}, \texttt{survive}, \texttt{uncertain}\}$ denotes the falsification verdict, C_{ref}^m and C_{unr}^m are the refuted and unresolved constraints, E_k^m denotes the retrieved evidence, and Q_{next}^m contains recommended follow-up queries. The falsification reports are then written back to the research state: $S_{k+1} = \mathcal{U}\left(S_k, \mathcal{H}_k, \{F_k^m\}_{m=1}^{M_k}\right)$.

Rejected candidates are removed, unresolved constraints are converted into new search targets, and supporting or contradicting evidence is preserved through `update_context`. Each context update defines a segment boundary, resulting in a trajectory composed of iterative exploration, falsification, and revision stages until the Searcher invokes `finish`.

Before agentic RL, we generate trajectories under the above protocol and apply rejection sampling to retain trajectories with correct answers, valid tool interactions, and sufficient evidence support. The retained trajectories are used for SFT warmup with the masked language-modeling objective:

$$\mathcal{L}_{\text{SFT}} = -\sum_i \sum_j m_{i,j} \log \pi_\theta(x_{i,j}|x_{i,<j}), \tag{3}$$

where $m_{i,j}$ masks environmental observations such as search results and webpage contents. This stage initializes the policy with basic capabilities in tool use, context updating, Searcher-Falsifier coordination, and candidate revision, providing a stable starting point for falsification-aware agentic RL.

3.4 Falsification-Aware Agentic Reinforcement Learning

Parallel Falsification-Aware Rollout At each RL iteration, given a query q, we sample n research trajectories in parallel from the current policy π_θ: $\mathcal{G}(q) = \{\tau_i\}_{i=1}^n$. Each trajectory executes the complete Searcher–Falsifier interaction loop online, with context compression enabled throughout the rollout. Parallel sampling explores diverse candidate hypotheses, evidence paths, and falsification outcomes, providing varied samples for subsequent policy optimization.

To improve the stability and efficiency of long-horizon online rollouts, we employ two asynchronous mechanisms. First, **asynchronous trajectory execution** allows different trajectories and their tool interactions to progress independently under trajectory-level time limits, preventing slow webpage visits or lengthy falsification rounds from blocking the entire rollout batch. Second, **asynchronous process evaluation** computes turn-level process scores once a trajectory is completed while the remaining trajectories continue generation. This design provides denser process supervision without substantially extending the critical rollout path.

Finally, each completed trajectory is divided into multiple segments at the boundaries defined by `update_context`.

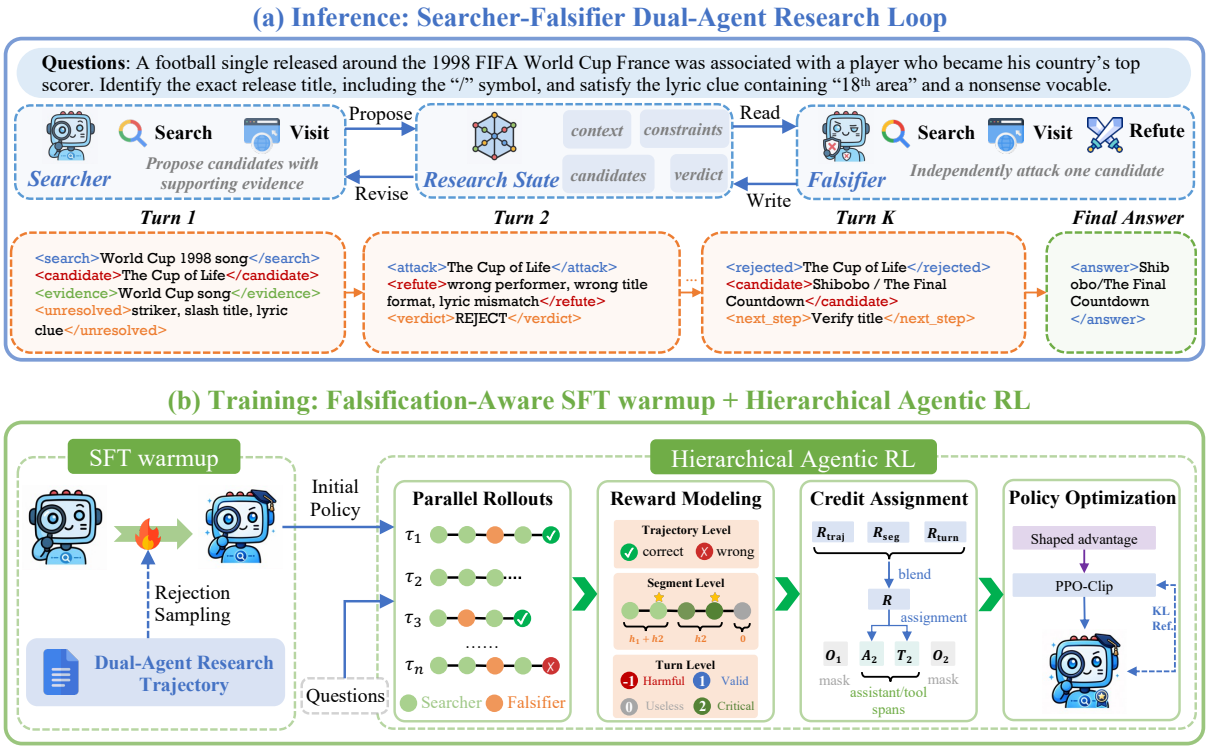


Figure 2: **Overview of FALCA**. The upper panel illustrates the Searcher-Falsifier dual-agent research loop for falsification-aware inference. The lower panel presents the training pipeline, consisting of SFT warmup, parallel rollouts, hierarchical reward modeling, credit assignment, and policy optimization.

These parallel trajectories and their segment-level structures serve as the training units for the hierarchical reward modeling and fine-grained credit assignment introduced below.

Hierarchical Reward Modeling and Credit Assignment Sparse outcome rewards provide limited process supervision for long-horizon deep search (Qi et al. 2025). A research trajectory may contain numerous search, verification, and state-update steps, while binary final-answer feedback cannot distinguish critical actions from redundant exploration within successful trajectories, nor identify locally valuable search behaviors in failed trajectories. We therefore model rewards at progressively finer granularities, from trajectory outcomes and segment-level research progress to local turn-level action quality, and assign these signals to policy-generated tokens.

The trajectory-level reward measures final task success. Given the predicted answer $\hat{y}$ and the ground-truth answer y^*, we define $R_{\text{traj}} = 1[\hat{y} \equiv y^*]$, where answer correctness is jointly determined by exact matching and an LLM judge. This reward provides the global task objective but does not distinguish the contributions of individual research steps.

We partition a complete trajectory into K research segments using `update_context`. For a multi-hop task, let h_j denote the segment in which the j-th subgoal is first completed, with $h_0 = 0$. The reward weight w_j associated with the j-th hop is uniformly distributed over the research stage between the previous and current hops:

$$R^k_{\text{seg}} = \sum_{j=1}^{J} \frac{w_j}{h_j - h_{j-1}} 1[h_{j-1} < k \le h_j]. \tag{4}$$

This design provides denser credit to the consecutive research stages that contribute to subgoal completion, while avoiding the propagation of later-hop rewards to unrelated earlier behaviors. Segments after the final hop receive no segment-level reward. To further assess local action quality, we use an LLM judge to score each turn: $s_t \in \{-1, 0, 1, 2\}$, corresponding to harmful, neutral, useful, and critical actions, respectively. The score is normalized as

$$R^t_{\text{turn}} = \frac{\text{clip}(s_t, -1, 2) + 1}{3}. \tag{5}$$

The evaluation considers not only retrieval relevance, but also whether an action acquires reliable evidence, identifies unmet constraints, falsifies an unsupported hypothesis, resolves evidence conflicts, or induces a meaningful research-state revision. Finally, we assign the three reward signals to their corresponding policy tokens. For a token x_i belonging to segment $k(i)$ and turn $t(i)$, we define its shaped advantage as

$$A^{\text{shape}}_i = \lambda_{\text{traj}} \frac{R_{\text{traj}}}{\sqrt{K}} + \lambda_{\text{seg}} R^{k(i)}_{\text{seg}} + \lambda_{\text{turn}} R^{t(i)}_{\text{turn}}, \tag{6}$$

where $\lambda_{\text{traj}} + \lambda_{\text{seg}} + \lambda_{\text{turn}} = 1$.

The trajectory-level reward is broadcast across the entire trajectory and normalized by $\sqrt{K}$ to prevent trajectories with more segments from receiving disproportionately large global credit. The segment-level reward is assigned to policy tokens within the corresponding research stage, while the turn-level reward is applied only to the associated turn span. Environmental observations, including search results and webpage contents, are fully masked. The resulting token-level shaped advantages are used for policy optimization in the following section.

Policy Optimization We optimize the policy using the token-level shaped advantages, without applying group-wise relative normalization. This choice is motivated by the sparse and nearly binary outcomes of long-horizon deep search. For difficult queries, trajectories sampled from the same prompt may receive identical final rewards, causing group-relative advantages to vanish. In contrast, the shaped advantages preserve learning signals from segment-level progress and turn-level action quality, even when all trajectories within a rollout group have the same outcome.

For token $x_{i,t}$ in the i-th trajectory, we define the importance ratio as

$$r_{i,t}(\theta) = \frac{\pi_\theta(x_{i,t}|x_{i,<t})}{\pi_{\theta_{\text{old}}}(x_{i,t}|x_{i,<t})}. \tag{7}$$

We then optimize the policy using a PPO-style clipped objective (Schulman et al. 2017):

$$\bar{r}_{i,t}(\theta) = \text{clip}(r_{i,t}(\theta), 1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}})\,. \tag{8}$$

$$\mathcal{L}_{\text{policy}}(\theta) = -\mathbb{E}_{i,t}\left[\min\left(r_{i,t}(\theta)A^{\text{shape}}_{i,t}, \bar{r}_{i,t}(\theta)A^{\text{shape}}_{i,t}\right)\right]. \tag{9}$$

Following the asymmetric clipping strategy used in DAPO (Yu et al. 2026), we adopt a slightly larger upper clipping bound, allowing the policy to more sufficiently increase the probabilities of high-value search, falsification, and research-state revision actions. To constrain policy drift from the supervised initialization, we further introduce token-level KL regularization with respect to a fixed SFT reference policy π_{ref}:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{policy}}(\theta) + \beta\mathbb{E}_{i,t}[D_{\text{KL}}(\pi_\theta(\cdot|x_{i,<t})\|\pi_{\text{ref}}(\cdot|x_{i,<t}))]. \tag{10}$$

This optimization preserves the exploration benefits of parallel rollouts while avoiding the loss of learning signals caused by degenerate rollout groups. The clipped objective and reference-policy regularization further stabilize policy updates during long-horizon agentic reinforcement learning.

4 Experiments

4.1 Experimental Setup

Datasets and Baselines We evaluate FALCA on five search-augmented reasoning benchmarks: **BrowseComp** and **DeepSearchQA** for long-horizon retrieval and evidence synthesis (Wei et al. 2025; Gupta et al. 2026); **GAIA** and **xbench-2510** for agentic planning, tool use, and multi-step reasoning (Mialon et al. 2024; Chen et al. 2025a); and **WideSearch-en** for broad-coverage retrieval and synthesis (Wong et al. 2025). We report Item-F1 for WideSearch-en, F1 for DeepSearchQA, and accuracy for the remaining benchmarks, with all scores reported in percentage points.

We compare with both closed-source and open-source baselines, including GPT-5 (Singh et al. 2025), Opus-4.6 (Anthropic 2026), Gemini-3.1-Pro (Google DeepMind 2026), Qwen3.5-397B (Qwen Team 2026), and recent search-oriented agents such as DeepMiner (Tang et al. 2025), WebDancer (Wu et al. 2026), WebSailor (Li et al. 2025), Tongyi-DeepResearch (Team et al. 2025b), MiroThinker (Team et al. 2025a), WebExplorer (Liu et al. 2025), LiteResearcher (Li et al. 2026), and AgentCPM-Explore (Chen et al. 2026). We also report Qwen3.5-4B and Qwen3.5-9B as backbone baselines.

Implementation Details We initialize the Searcher from an SFT-trained Qwen3.5-4B and optimize it with reinforcement learning, while the Falsifier uses the same backbone in an independent context. Turn-level process evaluation, final-answer judging, and webpage and context summarization are handled by Qwen3.5-35B-A3B. The `search` tool retrieves top-ranked titles, URLs, and snippets from a local HNSW index built with BGE-m3 embeddings (Chen et al. 2024; Malkov and Yashunin 2018). Training and rollouts use eight NVIDIA H100 80GB GPUs.

4.2 Main Results

As shown in Table 1, introducing the Falsifier-based harness yields clear overall gains while keeping the Qwen3.5-4B backbone unchanged. Compared with the default context-management harness, its average score increases from 51.8 to 55.5. This result suggests that, beyond collecting supporting evidence, periodically challenging candidates, checking unmet constraints, and injecting falsification reports into the research state can improve search reliability. This observation is consistent with our motivation: active falsification helps mitigate confirmation bias around early hypotheses and reduce the accumulation of incorrect candidates and unresolved contradictions over long-horizon interactions.

Under the same Falsifier-based harness, FALCA-4B improves the average score from 55.5 to 64.7, outperforming the falsification-untrained Qwen3.5-4B on all benchmarks. This indicates that merely introducing a Falsifier is insufficient to fully realize the benefits of falsification. SFT warmup teaches the Searcher the basic propose–falsify protocol, while agentic RL optimizes how it incorporates falsification reports, revises the research state, and redirects subsequent retrieval. Notably, FALCA-4B also surpasses Qwen3.5-9B under the same harness (64.7 vs. 57.9) and remains competitive with several 30B–32B search agents. These results suggest that falsification-aware training can be more parameter-efficient than simply scaling up the backbone.

FALCA-4B achieves its largest improvements on BrowseComp, xbench-2510, and DeepSearchQA, which require multi-step evidence aggregation, constraint verification, candidate elimination, and research-state revision. This pattern suggests that falsification is beneficial for constraint-intensive deep-search tasks. On WideSearch-en, which em-

Model	BrowseComp	GAIA	DeepSearchQA	xbench-2510	WideSearch-en
Default Context-Management Harness					
GPT-5	54.9	76.4	79.0	75.0	59.7
Opus-4.6	83.7	75.0	91.3	–	77.5
Gemini-3.1-Pro	85.9	80.6	93.3	53.0	66.4
Qwen3.5-397B	78.6	83.5	82.1	61.0	74.0
DeepMiner-32B	33.5	58.7	–	62.0	–
WebDancer-32B	3.8	51.5	–	39.0	–
WebSailor-32B	10.5	53.2	–	–	–
Tongyi-DeepResearch-30B	43.4	70.9	–	55.0	–
Qwen3.5-9B	34.2	65.0	65.5	51.0	17.5
MiroThinker-8B	31.1	66.4	36.7	34.0	–
WebExplorer-8B	15.7	32.0	17.8	53.7	–
LiteResearcher-4B	27.5	71.3	17.2	12.0	1.73
AgentCPM-Explore-4B	24.1	63.9	32.8	34.0	–
Qwen3.5-4B	32.7	65.0	62.6	45.0	53.9
Falsifier-Based Harness					
Qwen3.5-9B	41.4	68.0	65.0	48.0	**67.2**
Qwen3.5-4B	39.0	62.1	63.3	49.0	64.3
FALCA-4B	**56.0**	**72.8**	**70.8**	**58.0**	65.8

Notes. Best open-source results are bolded. A dash denotes an unavailable or pending result.

Table 1: Main experimental results on five search benchmarks.

phasizes broad-coverage information collection, the improvement is smaller, although FALCA-4B still achieves a strong absolute score of 65.8. Overall, FALCA-4B provides the largest benefits on falsification-intensive tasks while preserving strong broad-coverage retrieval performance.

4.3 Ablation Study

To assess the key inference and training components in FALCA, we construct three ablated variants. *w/o Falsifier* removes the Falsifier and its report injection, evaluating the role of active falsification and hypothesis revision. *w/o Agentic RL* retains only the SFT warmup, while *w/o Process-level Reward* removes the segment- and turn-level rewards and relies solely on trajectory-level feedback. All variants use the same backbone, training data, inference budget, and evaluation protocol. As shown in Figure 3, FALCA achieves the highest score of 64.7. Removing the Falsifier causes the largest drop to 52.8, highlighting the importance of seeking counterevidence, identifying constraint violations, and revising incorrect hypotheses. Without agentic RL, the score decreases to 60.5, indicating that SFT establishes the falsification protocol, while RL improves how the agent uses falsification feedback during multi-turn search. Removing process-level rewards reduces the score to 60.1, suggesting that terminal outcomes provide insufficient credit for intermediate evidence acquisition, candidate elimination, and research-state revision. Overall, the Falsifier provides the core inference mechanism, while agentic RL and process-level rewards enable it to be learned and executed more effectively.

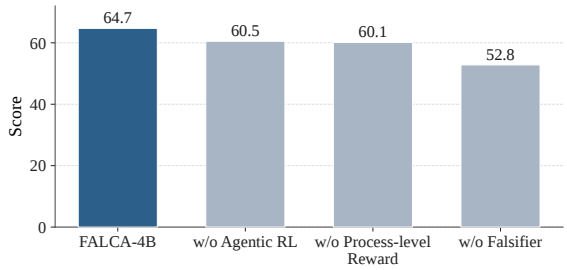


Figure 3: Ablation study of the key inference and training components in FALCA.

4.4 Behavioral Analysis

Falsification-Driven Revision We examine whether Falsifier reports induce actionable research-state revisions rather than merely provide additional verification. As shown in the real xbench-2510 trajectory in Figure 4, the injected report causes the Searcher to discard an incorrect hypothesis, revise its research state, and redirect retrieval toward the correct candidate. This case provides qualitative evidence that falsification functions as an actionable state transition in FALCA.

Long-Horizon Test-Time Scaling To study depth-based test-time scaling on BrowseComp, we fix the model and tool configuration and measure the cumulative number of correct trajectories and mean search calls at each round threshold T. As shown in Figure 5, 50% of correct trajectories finish within 20 rounds and 89% within 120 rounds. Extending the threshold to 200 rounds increases coverage only to 93%, while mean search calls rise from 4.3 to 22.9. These results demonstrate positive reasoning-depth scaling, but with diminishing returns beyond approximately 120–150 rounds.

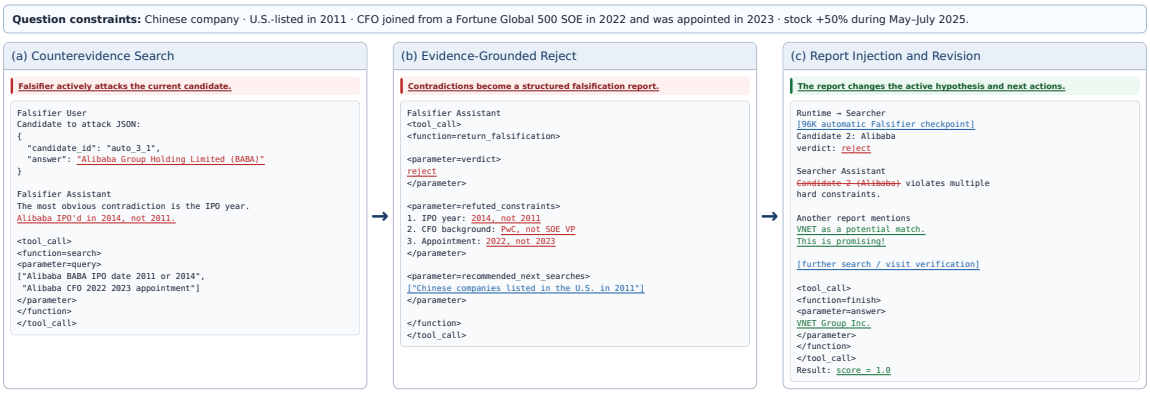


Figure 4: **A real falsification-driven revision trajectory from xbench-2510.** The Falsifier refutes Alibaba by identifying violations in its listing year, CFO background, and appointment date. After the report is injected, the Searcher adopts VNET as its new hypothesis, verifies the remaining constraints, and returns the correct answer.

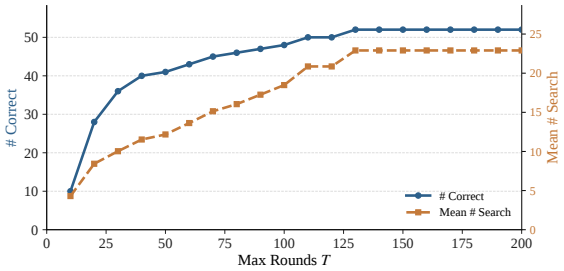


Figure 5: Long-horizon test-time scaling on BrowseComp.

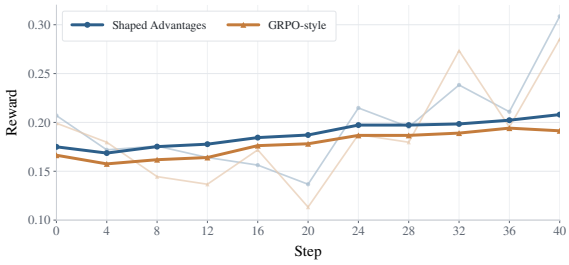


Figure 6: Training reward dynamics of shaped and GRPO-style advantages under degenerate rollout groups.

4.5 Training Dynamics and Analysis

Long-horizon deep search exhibits sparse terminal rewards. Complex queries require multiple rounds of retrieval, constraint verification, and evidence synthesis, while trajectories may terminate due to timeouts, interaction limits, or length truncation. Consequently, trajectories sampled for the same query often receive identical rewards; in our training, approximately 66% of rollout groups have zero within-group reward variance, weakening GRPO-style group-relative advantages.

In contrast, our shaped advantages combine trajectory-, segment-, and turn-level rewards to capture process-level differences within individual trajectories. As shown in Figure 6, the training reward of GRPO-style optimization reaches an early plateau, whereas shaped advantages exhibit a more sustained upward trend. This suggests that fine-grained process signals can continue to provide useful supervision even when trajectories share the same terminal outcome. As reported in Table 2, shaped advantages improve the average score from 61.2 to 64.7 and induce more turns, tokens, and search calls. These results suggest that shaped advantages preserve informative learning signals under degenerate rollout groups, supporting more persistent long-horizon exploration.

Setting	Avg.	Turns	Tokens	Search	Visit
Shaped Advantages	**64.7**	79.8	8.7M	63.6	14.9
GRPO-style	61.2	62.8	6.7M	47.7	14.0

Table 2: Comparison of task performance and search behavior between shaped and GRPO-style advantages.

5 Conclusion

We introduced FALCA, a falsification-aware reinforcement learning framework that formulates long-horizon deep search as an iterative propose–falsify process. FALCA combines independent Falsifiers for research-state revision with SFT warmup, parallel agentic rollouts and hierarchical credit assignment over trajectories, research segments, and individual actions. Experiments on five benchmarks demonstrate consistent improvements over backbone and harness baselines with strong parameter efficiency. Ablation studies identify the Falsifier as the primary source of performance gains and validate the contributions of agentic RL and process-level rewards. Training and behavioral analyses show that shaped advantages preserve learning signals under degenerate rollout groups, while falsification reports can redirect the Searcher away from plausible but incorrect hypotheses. Overall, our results suggest that reliable long-horizon deep search depends not only on searching deeper, but also on continually questioning and repairing the research state.

References

Anthropic. 2026. Claude Opus 4.6 System Card. https://www.anthropic.com/news/claude-opus-4-6.

Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; and Hajishirzi, H. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations*.

Chen, H.; Cong, X.; Fan, S.; Fu, Y.; Gong, Z.; Lu, Y.; Li, Y.; Niu, B.; Pan, C.; Song, Z.; et al. 2026. AgentCPM-Explore: Realizing Long-Horizon Deep Exploration for Edge-Scale Agents. *arXiv preprint arXiv:2602.06485*.

Chen, J.; Xiao, S.; Zhang, P.; Luo, K.; Lian, D.; and Liu, Z. 2024. Bge m3-embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 4(5).

Chen, K.; Ren, Y.; Liu, Y.; Hu, X.; Tian, H.; Xie, T.; Liu, F.; Zhang, H.; Liu, H.; Gong, Y.; et al. 2025a. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations. *arXiv preprint arXiv:2506.13651*.

Chen, M.; Sun, L.; Li, T.; Sun, H.; Zhou, Y.; Zhu, C.; Wang, H.; Pan, J. Z.; Zhang, W.; Chen, H.; Yang, F.; Zhou, Z.; and Chen, W. 2025b. ReSearch: Learning to Reason with Search for LLMs via Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 38.

Feng, L.; Xue, Z.; Liu, T.; and An, B. 2025. Group-in-Group Policy Optimization for LLM Agent Training. In *Advances in Neural Information Processing Systems*, volume 38.

Google DeepMind. 2026. Gemini 3.1 Pro Model Card. https://deepmind.google/models/model-cards/gemini-3-1-pro/.

Gou, Z.; Shao, Z.; Gong, Y.; Shen, Y.; Yang, Y.; Duan, N.; and Chen, W. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *The Twelfth International Conference on Learning Representations*.

Gupta, N.; Chatterjee, R.; Haas, L.; Tao, C.; Wang, A.; Liu, C.; Oiwa, H.; Gribovskaya, E.; Ackermann, J.; Blitzer, J.; et al. 2026. DeepSearchQA: Bridging the Comprehensiveness Gap for Deep Research Agents. *arXiv preprint arXiv:2601.20975*.

He, G.; Yang, Z.; Liu, J.; Xu, B.; Hou, L.; and Li, J. 2025. Webseer: Training deeper search agents through reinforcement learning with self-reflection. *arXiv preprint arXiv:2510.18798*.

Hu, M.; Fang, T.; Zhang, J.; Ma, J.-Y.; Zhang, Z.; Zhou, J.; Zhang, H.; Mi, H.; Yu, D.; and King, I. 2025. WebCoT: Enhancing Web Agent Reasoning by Reconstructing Chain-of-Thought in Reflection, Branching, and Rollback. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 5155–5173. Suzhou, China: Association for Computational Linguistics.

Jin, B.; Zeng, H.; Yue, Z.; Yoon, J.; Arik, S. O.; Wang, D.; Zamani, H.; and Han, J. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. In *The Second Conference on Language Modeling*.

Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.

Li, K.; Zhang, Z.; Yin, H.; Zhang, L.; Ou, L.; Wu, J.; Yin, W.; Li, B.; Tao, Z.; Wang, X.; et al. 2025. Websailor: Navigating super-human reasoning for web agent. *arXiv preprint arXiv:2507.02592*.

Li, W.; Qu, B.; Pan, B.; Zhang, J.; Liu, Z.; Zhang, P.; Chen, W.; and Zhang, B. 2026. Literesearcher: A scalable agentic rl training framework for deep research agent. *arXiv preprint arXiv:2604.17931*.

Liu, J.; Li, Y.; Zhang, C.; Li, J.; Chen, A.; Ji, K.; Cheng, W.; Wu, Z.; Du, C.; Xu, Q.; et al. 2025. Webexplorer: Explore and evolve for training long-horizon web agents. *arXiv preprint arXiv:2509.06501*.

Luo, X.; Zhang, Y.; He, Z.; Wang, Z.; Zhao, S.; Li, D.; Qiu, L. K.; and Yang, Y. 2025. Agent Lightning: Train ANY AI Agents with Reinforcement Learning. *arXiv preprint arXiv:2508.03680*.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, volume 36, 46534–46594. Curran Associates, Inc.

Malkov, Y. A.; and Yashunin, D. A. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4): 824–836.

Mialon, G.; Fourrier, C.; Wolf, T.; LeCun, Y.; and Scialom, T. 2024. Gaia: a benchmark for general ai assistants. In *International Conference on Learning Representations*, volume 2024, 9025–9049.

Nakano, R.; Hilton, J.; Balaji, S.; Wu, J.; Ouyang, L.; Kim, C.; Hesse, C.; Jain, S.; Kosaraju, V.; Saunders, W.; Jiang, X.; Cobbe, K.; Eloundou, T.; Krueger, G.; Button, K.; Knight, M.; Chess, B.; and Schulman, J. 2021. WebGPT: Browser-assisted Question-Answering with Human Feedback. *arXiv preprint arXiv:2112.09332*.

Popper, K. R. 1963. *Conjectures and Refutations: The Growth of Scientific Knowledge*. London: Routledge & Kegan Paul.

Press, O.; Zhang, M.; Min, S.; Schmidt, L.; Smith, N. A.; and Lewis, M. 2023. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 5687–5711.

Qi, Z.; Liu, X.; Iong, I. L.; Lai, H.; Sun, X.; Sun, J.; Yang, X.; Yang, Y.; Yao, S.; Xu, W.; et al. 2025. Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, volume 2025, 79791–79821.

Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. https://qwen.ai/blog?id=qwen3.5.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.

Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 36.

Singh, A.; Fry, A.; Perelman, A.; Tart, A.; Ganesh, A.; El-Kishky, A.; McLaughlin, A.; Low, A.; Ostrow, A.; Ananthram, A.; et al. 2025. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*.

Song, H.; Jiang, J.; Min, Y.; Chen, J.; Chen, Z.; Zhao, W. X.; Fang, L.; and Wen, J.-R. 2025. R1-Searcher: Incentivizing the Search Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2503.05592*.

Tang, Q.; Xiang, H.; Yu, L.; Yu, B.; Lu, Y.; Han, X.; Sun, L.; Zhang, W.; Wang, P.; Liu, S.; et al. 2025. Beyond turn limits: Training deep search agents with dynamic context window. *arXiv preprint arXiv:2510.08276*.

Team, M.; Bai, S.; Bing, L.; Chen, C.; Chen, G.; Chen, Y.; Chen, Z.; Chen, Z.; Dai, J.; Dong, X.; et al. 2025a. Mirothinker: Pushing the performance boundaries of open-source research agents via model, context, and interactive scaling. *arXiv preprint arXiv:2511.11793*.

Team, T. D.; Li, B.; Zhang, B.; Zhang, D.; Huang, F.; Li, G.; Chen, G.; Yin, H.; Wu, J.; Zhou, J.; et al. 2025b. Tongyi deepresearch technical report. *arXiv preprint arXiv:2510.24701*.

Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 10014–10037. Toronto, Canada: Association for Computational Linguistics.

Wei, J.; Sun, Z.; Papay, S.; McKinney, S.; Han, J.; Fulford, I.; Chung, H. W.; Passos, A. T.; Fedus, W.; and Glaese, A. 2025. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*.

Wong, R.; Wang, J.; Zhao, J.; Chen, L.; Gao, Y.; Zhang, L.; Zhou, X.; Wang, Z.; Xiang, K.; Zhang, G.; et al. 2025. Widesearch: Benchmarking agentic broad info-seeking. *arXiv preprint arXiv:2508.07999*.

Wu, J.; Li, B.; Fang, R.; Yin, W.; Zhang, L.; Wang, Z.; Tao, Z.; Zhang, D.-C.; Xi, Z.; Tang, R.; et al. 2026. Webdancer: Towards autonomous information seeking agency. *Advances in Neural Information Processing Systems*, 38: 120957–120985.

Xi, Z.; Liao, C.; Li, G.; Yang, Y.; Chen, W.; Zhang, Z.; Wang, B.; Jin, S.; Zhou, Y.; Guan, J.; Wu, W.; Ji, T.; Gui, T.; Zhang, Q.; and Huang, X. 2025. AgentPRM: Process Reward Models for LLM Agents via Step-Wise Promise and Progress. *arXiv preprint arXiv:2511.08325*.

Xia, S.; Wang, Y.; Huang, Z.; and Liu, P. 2026. Diagnosing and Mitigating Context Rot in Long-horizon Search. *arXiv preprint arXiv:2606.29718*.

Xia, Z.; Luo, K.; Qian, H.; and Liu, Z. 2025. Open data synthesis for deep research. *arXiv preprint arXiv:2509.00375*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

Yao, Y.; Huang, S.; Dai, E.; Tan, Z.; Duan, Z.; Jia, S.; Jiang, Y.; and Yang, T. 2026. ARC: Active and Reflection-driven Context Management for Long-Horizon Information Seeking Agents. 18644–18659.

Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Dai, W.; Fan, T.; Liu, G.; Liu, L.; et al. 2026. Dapo: An open-source llm reinforcement learning system at scale. *Advances in Neural Information Processing Systems*, 38: 113222–113244.

Zheng, Y.; Fu, D.; Hu, X.; Cai, X.; Ye, L.; Lu, P.; and Liu, P. 2025. DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-World Environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 414–431. Suzhou, China: Association for Computational Linguistics.

A Benchmark Details

A.1 Overall Coverage

We evaluate FALCA on five complementary benchmarks that cover diverse forms of search-augmented reasoning. BrowseComp and DeepSearchQA emphasize long-horizon information seeking, requiring persistent retrieval, multi-source evidence aggregation, and answer synthesis. GAIA evaluates general agentic capabilities, including planning, tool use, multimodal understanding, and multi-step reasoning, while xbench-2510 focuses more specifically on end-to-end deep search in the Chinese web environment. WideSearch-en complements these depth-oriented benchmarks by evaluating broad information seeking, where an agent must collect and organize a large number of atomic facts from a wide search space. Together, these benchmarks span constraint-intensive deep search, general tool-augmented reasoning, and broad-coverage information collection, enabling a comprehensive evaluation of search depth, constraint verification, research-state maintenance, and information coverage.

A.2 BrowseComp

BrowseComp consists of 1,266 open-domain questions designed to evaluate an agent's ability to persistently browse the web for hard-to-find and entangled information. Although its reference answers are typically short and easily verifiable, they are difficult to obtain through direct lexical matching with the original question. Solving these tasks often requires repeated query reformulation, integration of evidence distributed across webpages, and elimination of plausible candidates that satisfy only part of the given constraints. BrowseComp therefore provides a focused evaluation of persistent retrieval, candidate verification, and constraint-intensive reasoning.

A.3 GAIA

GAIA is a general-assistant benchmark comprising 466 real-world questions with unambiguous answers. Its tasks require combinations of logical reasoning, web browsing, multimodal information processing, file handling, computation, and external tool use. Unlike benchmarks devoted exclusively to web retrieval, GAIA evaluates whether an agent can identify the operations required by a task, organize them into a coherent multi-step plan, and maintain task-relevant information across heterogeneous interactions.

A.4 DeepSearchQA

DeepSearchQA contains 900 handcrafted prompts across 17 domains and evaluates whether agents can execute complex, multi-step search plans to produce exhaustive answer sets. Each task is structured as a causal search chain in which later discoveries depend on the successful completion of earlier steps. In addition to retrieving fragmented information from multiple sources, agents must perform entity resolution and deduplication, maintain context over long trajectories, and determine when the open-ended search space has been sufficiently covered. We report F1, which balances the precision of the predicted answer set against its coverage of the reference answers.

A.5 xbench-2510

xbench-DeepSearch evaluates end-to-end search capabilities, including planning, retrieval, reasoning, and information summarization. We use xbench-DeepSearch-2510, a refreshed 100-question version designed around the Chinese web environment. Its questions are expert-curated and typically require agents to integrate multiple constraints, distinguish among partially matching candidates, and synthesize evidence into a uniquely verifiable answer.

A.6 WideSearch-en

WideSearch evaluates broad information seeking rather than deep investigation of a single hidden answer. The full benchmark contains 200 manually curated tasks from more than 15 domains, equally divided between English and Chinese; we evaluate on its 100-question English subset, WideSearch-en. Each task requires the agent to collect a large set of independently verifiable atomic facts and organize them into a structured output. Its primary challenge is therefore to maximize information coverage while avoiding irrelevant, duplicated, or incorrect entries. We report Item-F1, which jointly measures item-level precision and recall over the collected information.

B Baselines

We compare FALCA with representative search agents spanning model scales from 4B to 32B parameters. These baselines cover diverse approaches to long-horizon information seeking, including challenging data synthesis, context management, interaction scaling, and agentic reinforcement learning. This comparison evaluates both absolute search performance and parameter efficiency. For external agents, we use results reported in their original papers or official technical reports only when the benchmark version and evaluation metric are compatible with ours; otherwise, the corresponding result is marked as unavailable.

B.1 Large-Scale Agents

We include four 30B–32B search agents. **DeepMiner-32B** combines reverse-constructed, high-difficulty search tasks with a dynamic context-window mechanism to support sustained long-horizon interactions. **WebDancer-32B** develops an end-to-end information-seeking agent through browsing-data construction, trajectory sampling, supervised fine-tuning, and reinforcement learning. **WebSailor-32B** targets complex information-seeking tasks through high-uncertainty data construction, rejection-sampling fine-tuning, and agentic reinforcement learning. **Tongyi-DeepResearch-30B** integrates agentic mid-training and post-training with scalable automatic data synthesis for long-horizon deep research.

B.2 Compact Agents

We further compare with four compact search agents. **MiroThinker-8B** studies interaction scaling, training the

model to benefit from deeper and more frequent agent–environment interactions. **WebExplorer-8B** constructs challenging search tasks through model-based exploration and iterative long-to-short query evolution, followed by supervised fine-tuning and reinforcement learning. **LiteResearcher-4B** introduces a lightweight virtual search environment to enable scalable and stable agentic RL for deep-research agents. **AgentCPM-Explore-4B** targets long-horizon exploration at the edge-model scale through model fusion, reward denoising, and contextual information refinement. The two 4B agents provide the most direct parameter-matched comparison with FALCA-4B.

C Implementation Settings

D Prompts

The complete prompts used in our experiments are provided in Figures 7–13. We use role-specific prompts to separate hypothesis construction from adversarial verification. The Searcher is instructed to maintain multiple candidate hypotheses, distinguish hard constraints from soft clues, gather URL-grounded evidence, and periodically externalize its research state through `update_context`. In contrast, the Falsifier follows a contradiction-first protocol: it independently attacks one candidate, checks whether the candidate violates the task constraints or requested answer type, and returns a structured verdict of `reject`, `survive`, or `uncertain`.

We additionally use auxiliary prompts for evidence processing, context management, and evaluation. The visit summarization prompt extracts goal-conditioned evidence from webpages while producing a compact summary for later context folding. When the context limit is reached, the context limit prompt requires the Searcher either to serialize the current research state or to terminate with a verified answer. Finally, the answer grading prompt extracts the predicted answer and evaluates its correctness against the reference answer under a standardized format.

E Tool Specifications

The complete function schemas are shown in Figures 14–18. The five tools support three stages of the propose–falsify research process. First, `search` and `visit` provide evidence acquisition. The former executes multiple complementary queries in one call, while the latter retrieves goal-conditioned evidence from one or more webpages.

Second, `update_context` serves as the central research-state transition interface. It serializes the current interaction history into an evidence-grounded context, an explicit constraint list, and a set of candidate hypotheses. When candidate hypotheses are provided, the runtime invokes independent Falsifiers and appends their reports to the refreshed research state, thereby connecting context compression with active falsification.

Finally, `finish` and `refute` define the terminal actions of the Searcher and Falsifier, respectively. The Searcher uses `finish` to return a concise answer together with supporting evidence and a confidence estimate. The Falsifier uses `refute` to return a candidate-level verdict, constraint status, relevant sources, and recommended follow-up searches. Together, these tools provide a structured interface for evidence acquisition, research-state revision, and answer synthesis.

F Case Studies

Figures 19 and 20 illustrate how falsification changes the research trajectory rather than merely verifying the final answer. Each case follows three stages: the Falsifier first searches for counterevidence against the current candidate, then returns a structured verdict with violated constraints and recommended next searches, and finally injects the report into the Searcher context to trigger hypothesis revision.

Figure 19 shows a sportsperson-identification task with multiple biographical, temporal, and geographic constraints. The initial candidate, Joseph G Abraham, superficially matches the award and gold-medal clues, but violates two decisive conditions: he was born after Indian independence, and his gold-medal event was held in Guangzhou rather than a capital city. The Falsifier converts these contradictions into a structured rejection report and recommends searches centered on pre-independence-born Indian gold medalists. After receiving the report, the Searcher discards both Joseph G Abraham and a nearby distractor, redirects the search, and identifies Tulsidas Balaram as the verified answer.

Figure 20 presents an author-identification task in which the initial candidate, Mark Eveleigh, is strongly associated with the relevant article topic but fails the biographical constraints. In particular, his surname is not hyphenated and available biographies do not support a prior legal career. The Falsifier therefore rejects the candidate and proposes more discriminative searches combining the publication, hyphenated surname, and former-lawyer attributes. The resulting context revision leads the Searcher to Ishay Govender-Ypma, after which the Searcher returns the required first name, Ishay.

Searcher System Prompt

Search intensity is high. You are the **main Proposer** in an adversarial Falsifier search loop. Your task is to find the exact answer with evidence, not merely a plausible answer.

Core policy:

- Maintain candidate answers when evidence permits. Track each candidate's support, unresolved constraints, and possible contradictions.
- Optimize for candidate recall before candidate precision. Keep several plausible candidates alive early, including weak or partially supported candidates, instead of collapsing to one anchor too soon.
- Explore multiple independent search frames when the task is ambiguous: exact-phrase retrieval, clue-to-document retrieval, entity/author intersection, source/domain-specific search, and broad synonym/paraphrase search. If one frame stalls, switch frames.
- Track assumptions you added that are not stated in the question, such as field, nationality, gender, time period, source type, or answer type. Actively search against these assumptions before trusting them.
- Separate hard constraints from soft clues. A hard contradiction should eliminate or sharply downgrade a candidate.
- Use discriminative searches that can refute candidates, separate close candidates, or verify unresolved hard constraints.
- Use `visit` for decisive evidence; snippets alone are not enough for final claims or candidate rejection.
- Use `update_context` as a checkpoint: pass dense "context", "constraints", and a non-empty "candidates" list whenever any plausible candidate or answer frame exists. The runtime will run independent Falsifier agents and append their attack reports to the refreshed context.
- Avoid "candidates: []" unless you have explicitly tried at least two independent recall routes and still have no plausible candidate. If unsure, include weak candidates with clear unresolved constraints.
- Falsifier reports are strong evidence to integrate. Treat "reject" as a serious blocker unless later direct evidence overturns it; treat "uncertain" as a warning that more verification or candidate recall is needed; treat "survive" as useful only when hard constraints are directly supported.
- Do not finish with a candidate rejected or left uncertain by a Falsifier unless later evidence directly resolves the issue.
- Never simulate tool outputs. Always call the actual tools.

Tools:

... {tool_des} ...

Figure 7: System prompt for the Searcher agent.

Searcher User Prompt

Question: {question}

Start with a search tool call. Build evidence, constraints, and candidates as you go.

Before finishing:

- every critical claim must have a cited URL;
- unresolved contradictions must be resolved or explicitly judged non-decisive;
- candidate answers should have survived direct falsification or later evidence must explain why a rejection/uncertainty was wrong;
- the final answer must match the answer type and field asked by the question, not a nearby affiliation, source, person, or related entity.

When context is long, search stalls, or candidate analysis reaches a milestone, call `update_context` with "context", "constraints", and "candidates". Prefer passing 2-5 plausible candidates or explicit answer frames; use "[]" only after explicit recall attempts fail.

When the answer is sufficiently verified, call `finish` with a concise "answer", JSON-array "evidences", and "confidence".

Do not answer from memory and do not write text after a tool call.

Figure 8: User prompt for the Searcher agent.

Falsifier System Prompt

You are an **adversarial Falsifier agent** for Deep Search.
Your primary task is to attack one candidate hypothesis proposed by the main agent. Search for evidence that the candidate violates the question's hard constraints, answer type, source field, or implied entity role. Do not choose the final answer and do not help the candidate first.
Also attack the main agent's framing when relevant. If the candidate seems to come from a wrong assumption, wrong answer field, nearby entity drift, or too-narrow search space, report that clearly and recommend alternative recall searches.
Tools:
... {tool_des} ...
Rules:

- Use the actual `search`/`visit` tools. Never invent tool outputs.
- Search for refuting evidence before supporting evidence.
- Visit authoritative pages for decisive claims; snippets alone are not enough for rejection.
- Distinguish hard contradiction from ambiguity or missing evidence.
- Check the original question's requested answer type carefully. A related person, institution, article, affiliation, event, or source is a refutation if the question asks for a different field.
- "reject" when a hard constraint is contradicted, the answer type/field is wrong, or a better-supported interpretation makes the candidate untenable.
- "survive" only when the candidate has direct support for the key hard constraints. Do not use "survive" merely because no contradiction was found.
- Use "uncertain" when evidence is incomplete, indirect, snippet-only, or when the candidate might be right but key constraints remain unverified.
- For no-answer or "not found" candidates, aggressively search for one counterexample. Missing search results are not enough for "survive".
- Return exactly one `Refute` call when done.
- The report fields are: candidate_id, candidate_answer, verdict, refuted_constraints, supporting_constraints, unresolved_constraints, refuting_sources, supporting_sources, recommended_next_searches, summary.
- Allowed verdict values: reject, survive, uncertain.
- Do not output text after a tool call.

Figure 9: System prompt for the Falsifier agent.

Falsifier User Prompt

Original question:
`{question}`
Compressed main-agent context:
`{context}`
Constraint list JSON:
`{constraints_json}`
Candidate to attack JSON:
`{candidate_json}`
Attack this single candidate. Look for direct evidence that violates the hard constraints, answer type, source field, or makes another interpretation more likely. If you find a decisive contradiction or wrong-field answer, return `reject`. If contradiction search fails and key constraints are directly supported, return `survive`. If evidence is incomplete, indirect, or only shows that searches failed, return `uncertain`. Put alternative candidates, recall routes, and discriminative next searches in `recommended_next_searches`.

Figure 10: User prompt for candidate falsification.

Visit Summarization Prompt

Please process the following webpage content and user goal to extract relevant information:
Webpage Content: `{webpage_content}`
User Goal: `{goal}`
Task Guidelines:

- **Content Scanning for Rationale**: Locate the specific sections/data directly related to the user's goal within the webpage content.
- **Key Extraction for Evidence**: Identify and extract the most relevant information; never miss important information; output the full original context as far as possible; prefer a longer `evidence` field when the page contains multiple relevant details.
- **Summary Output for Summary**: Organize into a detailed summary with logical flow, preserving names, dates, institutions, locations, titles, and relationships whenever they help the goal.
- **Compact Summary for Folding**: Write a `compact_summary` of 120–180 words for later context folding. It must be dense, structured, and preserve decisive entities, dates, locations, relationships, evidence polarity, and the page's contribution to the user goal.
- If the page contains several independently useful facts, include all of them instead of compressing too aggressively.

Final output (JSON fields): `rational`, `evidence`, `summary`, `compact_summary`.

- `evidence`: can be long; preserve original wording/context as much as possible.
- `summary`: can be multiple paragraphs; should not be artificially shortened.
- `compact_summary`: 120–180 words, formatted for reuse after context folding.

Figure 11: Prompt for goal-conditioned webpage summarization.

Context Limit Prompt

The context limit has been exceeded for the task. You **MUST** either update the context now or finish now. If the answer is not fully verified, call `update_context` with the best current constraints and candidates so Falsifier agents can attack them. Prefer weak candidates or answer frames over `candidates: []`. If a Falsifier rejected or questioned the leading candidate, finish only if later direct evidence resolves that objection.
Output requirements (STRICT):

- Output exactly one tool call.
- The only allowed tool call is `update_context` or `finish`.

Figure 12: Prompt for handling context-limit events.

Answer Grading Prompt

Judge whether the following `[response]` to `[question]` is correct or not based on the precise and unambiguous `[correct_answer]` below.
`[question]: {question}`
`[response]: {response}`
Your judgement must be in the format and criteria specified below:
extracted_final_answer: The final exact answer extracted from the `[response]`. Put `None` if there is no exact final answer to extract.
`[correct_answer]: {correct_answer}`
reasoning: Explain why the extracted final answer is correct or incorrect based on `[correct_answer]`, focusing only on meaningful differences. Do not comment on background, do not solve the problem, and do not argue for any answer different from `[correct_answer]`.
correct: Answer `yes` if `extracted_final_answer` matches `[correct_answer]`, or is within a small margin of error for numerical problems. Answer `no` otherwise, i.e., if there is any inconsistency, ambiguity, non-equivalency, or if the extracted answer is incorrect.
confidence: The extracted confidence score between 0% and 100% from `[response]`. Put 100 if no confidence score is available.

Figure 13: Prompt for final-answer correctness evaluation.

Tool: search

```
{
  "type": "function",
  "function": {
    "name": "search",
    "description": "Performs batched web searches: supply an array 'query'; the tool
        retrieves the top 10 results for each query in one call",
    "parameters": {
      "type": "object",
      "properties": {
        "query": {
          "type": "array",
          "items": { "type": "string" },
          "description": "Array of query strings. Include multiple complementary search
              queries in a single call."
        }
      },
      "required": ["query"]
    }
  }
}
```

Figure 14: Function specification of the `search` tool.

Tool: visit

```
{
  "type": "function",
  "function": {
    "name": "visit",
    "description": "Visit webpage(s) and return the summary of the content.",
    "parameters": {
      "type": "object",
      "properties": {
        "url": {
          "type": ["string", "array"],
          "items": { "type": "string" },
          "minItems": 1,
          "description": "The URL(s) of the webpage(s) to visit. Can be a single URL or
              an array of URLs."
        },
        "goal": {
          "type": "string",
          "description": "The goal of the visit for webpage(s)."
        }
      },
      "required": ["url", "goal"]
    }
  }
}
```

Figure 15: Function specification of the `visit` tool.

Tool: update_context

```
{
  "type": "function",
  "function": {
    "name": "update_context",
    "description": "Compress the current research state and trigger the adversarial Falsifier checkpoint. Call this tool only by
        itself when context is long, the search state is cluttered, or you have reached a meaningful candidate-analysis
        milestone. `context` must be dense and evidence-grounded. `constraints` and `candidates` must be valid JSON arrays;
        when candidates is non-empty, independent Falsifier agents attack them and their reports are appended to the refreshed
        context.",
    "parameters": {
      "type": "object",
      "properties": {
        "context": {
          "type": "string",
          "description": "Dense, lossless research state with confirmed facts, URLs, constraints, candidates, failed paths, and
              next steps."
        },
        "constraints": {
          "type": "array",
          "description": "Task constraints and their current evidence status.",
          "items": {
            "type": "object",
            "properties": {
              "constraint_id": { "type": "string" },
              "constraint": { "type": "string" },
              "type": { "type": "string", "description": "hard or soft" },
              "status": { "type": "string", "description": "verified, contradicted, unresolved, or unknown" },
              "evidence": { "type": "string" },
              "url": { "type": "string" }
            },
            "required": ["constraint"]
          }
        },
        "candidates": {
          "type": "array",
          "description": "Candidate hypotheses for adversarial falsification. Use [] if none exists yet.",
          "items": {
            "type": "object",
            "properties": {
              "candidate_id": { "type": "string" },
              "answer": { "type": "string" },
              "supporting_evidence": {
                "type": "array",
                "items": {
                  "type": "object",
                  "properties": {
                    "evidence": { "type": "string" },
                    "url": { "type": "string" }
                  }
                }
              },
              "unresolved_constraints": {
                "type": "array",
                "items": { "type": "string" }
              },
              "status": { "type": "string" },
              "notes": { "type": "string" }
            },
            "required": ["answer"]
          }
        }
      },
      "required": ["context", "constraints", "candidates"]
    }
  }
}
```

Figure 16: Function specification of the `update_context` tool.

Tool: finish

```
{
  "type": "function",
  "function": {
    "name": "finish",
    "description": "Return the final result when you have a definitive answer. This
        function signals that your research is complete and you're ready to present
        the final answer to the user.",
    "parameters": {
      "type": "object",
      "properties": {
        "answer": {
          "type": "string",
          "description": "A succinct, final answer to the user's query"
        },
        "evidences": {
          "type": "array",
          "description": "Array of evidence objects supporting the final answer. Each
              piece of evidence must cite exactly one url.",
          "items": {
            "type": "object",
            "properties": {
              "evidence": {
                "type": "string",
                "description": "The specific verified fact or data point (one sentence)"
              },
              "url": {
                "type": "string",
                "description": "The source URL of the evidence"
              }
            },
            "required": ["evidence", "url"]
          }
        },
        "confidence": {
          "type": "string",
          "description": "Your confidence score between 0% and 100% for your answer"
        }
      },
      "required": ["answer", "evidences", "confidence"]
    }
  }
}
```

Figure 17: Function specification of the `finish` tool.

Tool: refute

```
{
  "type": "function",
  "function": {
    "name": "refute",
    "description": "Return the final falsification report for the single assigned
        candidate. This does not answer the original question.",
    "parameters": {
      "type": "object",
      "properties": {
        "candidate_id": { "type": "string" },
        "candidate_answer": { "type": "string" },
        "verdict": { "type": "string", "enum": ["reject", "survive", "uncertain"] },
        "refuted_constraints": { "type": "array", "items": { "type": "object" } },
        "supporting_constraints": { "type": "array", "items": { "type": "object" } },
        "unresolved_constraints": { "type": "array", "items": { "type": "object" } },
        "refuting_sources": { "type": "array", "items": { "type": "object" } },
        "supporting_sources": { "type": "array", "items": { "type": "object" } },
        "recommended_next_searches": { "type": "array", "items": { "type": "string" }
            },
        "summary": { "type": "string" }
      },
      "required": ["candidate_answer", "verdict", "summary"]
    }
  }
}
```

Figure 18: Function specification of the `refute` tool.



Figure 19: **Case study of constraint-based candidate rejection.** The Falsifier identifies decisive biographical and geographic contradictions in the initial candidate, returns a structured rejection report, and redirects the Searcher to the correct answer, Tulsidas Balaram.

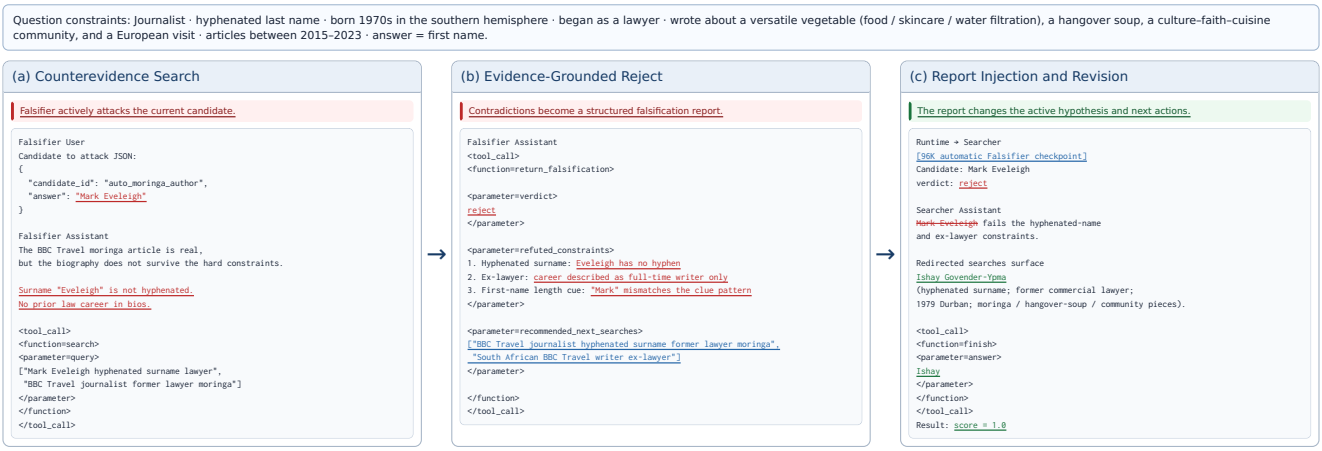


Figure 20: **Case study of biography-grounded search revision.**The Falsifier rejects a topic-matching but biographically inconsistent candidate and redirects the Searcher to Ishay Govender-Ypma.